

University of Cape Town

STA4010W

Analytics

Assessment 1: Applied Supervised Learning

Author:
Ethon Felix
Beno Tiroyakgosi

Student Number:
FLXETH002
TRYBEN001

Contents

Introduction **1**

1 Creating Target Variable **1**

2 Model Development and Selection **2**
Modelling Approach and Hyperparameter Search Space 2
Regularised Elasticnet Logistic Regression 2
Random forest 3
Gradient Boosting Machine 3
K-Nearest Neighbors 4
Cross-Validation at $\tau = 0.5$ 4
Regularised Elasticnet Logistic Regression 4
Random forest 5
Gradient Boosting Machine 6
K-Nearest Neighbors 7
Model Comparison 7

3 Threshold Optimisation **8**

4 Interpretation and Error Analysis **9**
Variable Importance Plots 9
Partial Dependence Plots 10
Missclassifcations 11

Introduction

The story of Airbnb in Cape Town is a textbook illustration of demand and supply dynamics. The city attracts tourists from all over the globe who come to experience it's attractions. This has resulted in a rise in short term rental activity to accommodate these visitors. These short term rentals have proven to be profitable and more flexible for property owners, compared to having long term leases. Consequently, Airbnb listed properties have become ubiquitous. Unfortunately, this adversely affects local residents and their livelihoods. The increased demand for housing has lead to high rental costs and shortage of affordable housing. Forcing residents to move to outskirts of the city in pursuit of affordable housing or accept high prices for the sake of convenience.

This report aims to take a supervised learning approach to model this phenomenon. Using Airbnb listings data, properties will be classified as having either high or low availability, which was one of three classification criteria to be selected from. A threshold defining these categories will be defined based on the number of days per annum the house is listed. For this binary classification problem, four modelling approaches will considered: a regularised elastic-net logistic regression, a random forest, an XGBoost model and a k-nearest neighbours model. The dataset will be randomly split into training (80%) and testing (20%) subsets. Thereafter, cross-validation will be employed for hyper-parameter tuning, and classification initially performed at a threshold of $\tau = 0.5$. A final model will be selected after assessing classification metrics. The selected model's predictive performance will be improved by tuning the threshold τ. The most influential features will be investigated by use of variable importance plots and partial dependence plots. Furthermore, misclassification will be analysed with the goal of pattern recognition. The objective is to demonstrate sound modelling practices rather than maximise predictive performance. This analysis was conducted in R.

1 Creating Target Variable

High availability was chosen as the response variable because it shows how actively a property participates in the short-term rental market.The feature `availability_365` tells us how many days a listing is kept open for booking over the next year. Listings that are almost always available are likely professionally managed and consistently rented out, meaning they are less likely to be part of the long-term rental market. By focusing on high availability, we can see which properties are continuously used for short-term rentals and how this affects access to long-term housing in Cape Town. This helps us understand how individual features relate to bigger issues, like the concentration of Airbnb rentals in certain neighbourhoods and have policy implications.

Figure 1 shows the histogram of availability_365. This plot shows a large number of listings available for most of the year which is indicative of these properties being used mainly for short-term rentals, which points to a clear commercial purpose rather than casual or occasional hosting. The threshold selected for classification into the target variable `HighAvailability` is 183 days which is equivalent to half a year. This threshold was selected because listings that are available for over half a year are properties that would require management, cleaning services and probably advertising. Therefore, these are most likely to be professional or semi-professional short term rental properties. These properties would be owned with aim of long-term financial gain from a premium charged to tourists.

Ideally the threshold would have been set to 3 months (90 days), as such properties share the same characteristics discussed above. However, that threshold would result in very high data imbalance. The threshold of 183 days resulted in a split of 33% and 67% for low and high availability respectively in the training set. The test set had a similar split of 32% and 68% respectively.

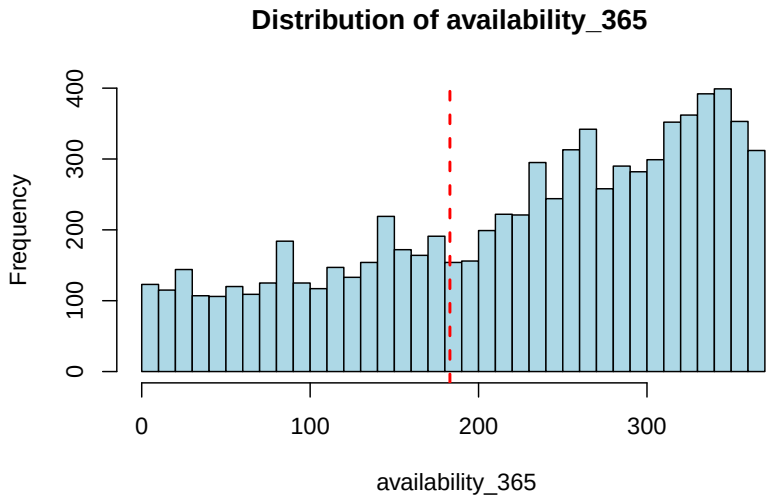


Figure 1: Histogram of the number of days listings are available in the next year with threshold value of 180 days used for classification represented by the red line.

In addition to `availability_365` -used to define the binary variable- a number of features were removed before conducting modelling. Features containing personal information about the host were removed as they provide no benefit for modelling. These are: `id`, `host_id`, `host_since`, `host_verifications`.The variables `availability_30`, `availability_60`, `availability_90` were removed to avoid data leakage as they contain information about high availability. Lastly, The variable `neighbourhood_ward` was removed as it provides the same information as `latitude` and `longitude`. Furthermore, it is recognised as a factor during predictions using the test set which is problematic given there are levels not observed during training.

2 Model Development and Selection

Modelling Approach and Hyperparameter Search Space

Regularised Elasticnet Logistic Regression

Logistic regression is a linear model used for binary classification, where the response variable takes two possible outcomes. Logistic regression models the log-odds of the probability of the positive class as a linear combination of the predictors, $\text{logit}(p) = \log \frac{p}{1-p} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$. The model produces predicted probabilities that can be converted into class labels using a threshold ,commonly 0.5. The parameters of this model will be estimated using elasticnet regularisation.

Elasticnet regularisation combines L_1 and L_2 regularisation to estimate regression parameters. When parameters are correlated (multicollinearity), this can make the model coefficients very large or have signs which do not show the true nature of the relationship with the response varibale. L_2 regularisation introduces a penalty of $\lambda \sum_{i=1}^{p} \beta_i^2$ to the maximum likelihood estimation of parameters. It does this subtracting this penalty from the log-likelihood, shrinking large coefficients and ensuring that the most important features have greater influence on the model's predictions. L_1 regularisation performs variable selection by shrinking unimportant features to zero and creating a model with less features. This differs from L_2 regularisation which can only reduce the coefficients close to zero and keeps the features in the model. The penalty term for L_1 regularisation is $\lambda \sum_{i=1}^{p} |\beta_i|$ and is administered in the same way as L_2. The penalty term has a weight parameter $\lambda \geq 0$, which determines the strength of the regularisation. In both methods, the intent is to maximise the log-likelihood given the penalty term introduced. Elasticnet is a regularisation method that combines L_1 and L_2 regularisation by defining the penalty term as

$\lambda[\ (1-\alpha)\sum_{i=1}^{p}\beta_i^2\ +\ \alpha\sum_{i=1}^{p}|\beta_i|\ ]$, where $\alpha \in [0,1]$, determines the contribution of the L1 and L2 penalties. Therefore, the estimated regression coefficients maximise log-likelihood - penalty.

The hyperparameter search spaces are: $\alpha \in \{0, 0.1, 0.2, \ldots, 1\}$ and $\lambda \in \{10^4, 10^{3.526}, \ldots, 10^0\}$. For α, a grid from 0 to 1 in increments of 0.1 allows the model to explore the full range of possible mixtures while keeping the computational cost manageable.The regularisation parameter λ affects the penalty multiplicatively, so meaningful changes in model complexity occur when λ is scaled by a factor rather than increased by a fixed amount. Therefore, λ was searched over a logarithmically spaced grid between 0 and 1, which allows both very small and relatively large levels of shrinkage to be explored efficiently.

Random forest

A random forest is a non-linear model used for classification and regression tasks. In this project it will be used for classification. A large number of decision trees are trained using bootstrap samples. Given p features, the decision trees split the p-dimensional feature space, where at each split a random subset of d features is selected. The splitting criterion aims to split the feature space into homogeneous regions, and to do this, the Gini index was employed in this project. The Gini index defined as: $G = \sum_{m=1}^{M}\sum_{k=1}^{K}\hat{p}_{mk}(1-\hat{p}_{mk})$, measures the the level of class mixture within each region, and the overall impurity is obtained by summing these impurity measures across the regions. Here k is the class and m is the node/leaf. The selected split is the one that reduces this index the most. These splits continue until a minimum node or leaf size is reached, at which point predictions are made based on the class concentration within the terminal node.

For a given observation, the final predicted class is the one most frequently selected across all trained trees.That is, $\hat{C}_{RF} = \textbf{mode}\{\hat{C}_1, \hat{C}_2, \hat{C}_3, \ldots \hat{C}_B\}$, where $\hat{C}$ is the predicted class and B is the number of trees. Random forests reduce correlation between trees by considering a random subset of predictors at each split. Without this random selection, the trees would tend to split on the same dominant predictors and therefore become highly similar. The aggregation of predictions across many such trees leads to more stable and reliable predictions.

The hyperparameters investigated are number of parameters randomly selected (d) and and minimum node size(s). The hyperparameter search space for number of randomly selected features is: $d \in \{2, 3, 4 \ldots, 23\}$. Examining the entire set of possible splits was chosen to explore all iterations, which allows for a range of different and uncorrelated trees. The hyperparameter space for minimum node size is, $s \in \{5, 10, 20\}$. Values below 5 would produce highly over-fitted models and values over 20 would be too simple and predictions would be based on a large number of points. The search space was limited to three values due to computational constraints and to avoid redundancy that may arise from evaluating a large numberof similar hyperparameter values.The model uses 500 trees, a number large enough to ensure reliable predictions. Increasing it further doesn't noticeably improve performance.

Gradient Boosting Machine

XGBoost is an advanced implementation of gradient boosted decision trees designed for speed and performance. While Random Forests build trees in parallel using bagging, XGBoost builds trees sequentially using boosting. Each new tree is trained to predict the residuals (errors) of the preceding ensemble, effectively "learning" from the mistakes of the previous iterations.

The model minimizes a regularized objective function that combines a loss function (Bernoulli loss for classification) and a penalty term to control model complexity:

$$\mathcal{L}(\phi) = \sum_{i=1}^{n}\ell(y_i, \hat{y}_i) + \sum_{m=1}^{M}\left(\gamma J_m + \frac{\lambda}{2}\sum_{j=1}^{J_m}w_{jm}^2\right)$$

where J_m is the number of leaves in tree m and w_{jm} are the leaf weights. The parameters γ and λ (and optionally an L_1 penalty) explicitly control tree complexity and shrink leaf predictions, thereby reducing the risk of overfitting.

Hyperparameter Search Space: The hyperparameter space was explored using a grid search over three key dimensions:

Learning Rate (η / shrinkage): Evaluated at $\eta \in \{0.01, 0.05, 0.1\}$. This scales the contribution of each new tree. Smaller values (0.01) require more trees but are more robust to overfitting, while larger values (0.1) speed up learning but risk overshooting the optimal solution.

Maximum Tree Depth: Evaluated at depth$\in \{5, 7, 10\}$. This controls the complexity of individual trees and their ability to capture high-order interactions. A depth of 10 allows the model to identify intricate non-linear relationships, such as the spatial effects of latitude combined with host listing counts.

Number of Boosting Iterations (n.trees): Evaluated at {250,500,750,1000}. This determines the total number of sequential trees. This accounts for increasing iterations beyond a certain point (e.g., 500 at a 0.05 shrinkage) which can lead to a decrease in cross-validation ROC as the model begins to overfit the training data.

K-Nearest Neighbors

The k-Nearest Neighbors algorithm is a non-parametric, instance-based learning method used for classification. Unlike logistic regression, kNN does not assume a functional form for the relationship between predictors and the response. Instead, it classifies an observation by identifying the k closest points in the training dataset based on a distance metric, typically the Euclidean distance, defined as $d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{p}(x_i - y_i)^2}$.

The predicted probability of an observation belonging to the "High" availability class is the proportion of its k neighbors that belong to that class:

$$\hat{p} = \frac{1}{k} \sum_{i \in \mathcal{N}_0} I(y_i = \text{"High"})$$

where $\mathcal{N}_0$ is the neighborhood of k points. A significant prerequisite for kNN is feature scaling; since the algorithm relies on distance, variables with larger magnitudes would disproportionately influence the model. In this project, all predictors were centered and scaled to ensure each feature contributes equally to the distance calculation.

Hyperparameter Search Space: The primary hyperparameter investigated is the number of neighbors, k. The search space was defined as k$\in \{1, 3, 5, \ldots, 51\}$. A small k (e.g., k=1) allows the model to capture local patterns but makes it highly sensitive to noise and outliers, leading to high variance. Conversely, a very large k smooths the decision boundary, potentially introducing high bias by over-generalizing. The range up to 51 was selected to identify the optimal balance where the cross-validated error is minimized, ensuring the neighborhood is large enough to be robust but small enough to remain local.

Cross-Validation at $\tau = 0.5$

Regularised Elasticnet Logistic Regression

This section presents the cross-validation results and the selection of model parameters for all four models used. Figure 2 shows cross-validation plot for the elastic-net logistic regression. This plot shows Receiver Operating Characteristic - Area Under the Curve (ROC(AUC)) values for models with different combinations of α and λ. The model with the highest ROC(AUC)is shown by the vertical red dashed line. This model has alpha = 0 and log-lambda = -1.9 (lambda = 0.013). This model is a ridge regression model (pure L_2 regularisation). A simpler model with very similar performance is shown by the vertical blue dashed line. This model has alpha = 0.5 and log-lambda = -2.9 (lambda = 0.001). This model performs variable selection in addition to handling correlation in the variables. Thus, this simpler model which considers sparsity and stability is selected.

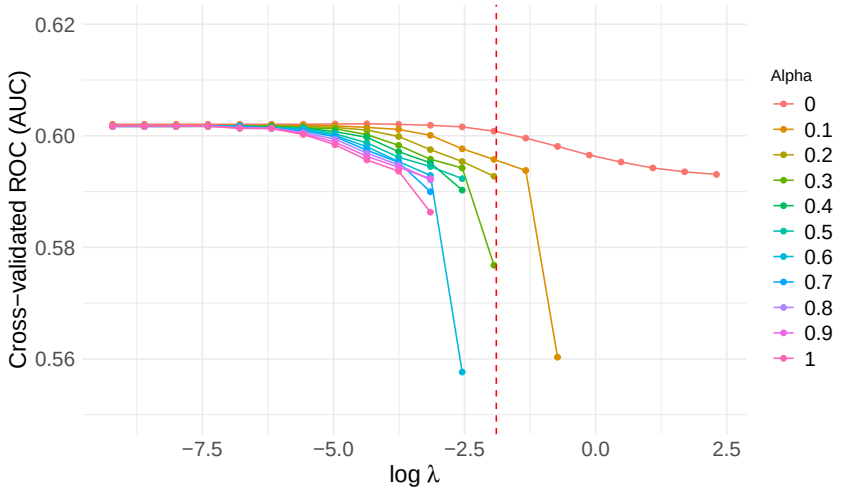


Figure 2: Cross-validation plot for elasticnet logistic regression using hyperparameters alpha and lambda. Cross-validation metric metric is ROC(AUC) and the best performing model by this metric is shown by the red vertical line. The best model has alpha = 0 and log-lambda = -1.9 (lambda = 0.013). The model selected is shown with the blue vertical line and has alpha = 0.5 and log-lambda = -2.9 (lambda = 0.001).

Random forest

Figure 3 shows the cross-validation results for the random forest. From this plot, it is observed that the best performing model has minimum node size (s) of 5 and randomly selects 6 (d) features when splitting. A simpler model with negligible difference in ROC(AUC) has $s = 20$ and $d = 4$. This selection is based on the node size because the model with $s = 20$ will generalise better while having similar predictive performance.Therefore, the use of a more complex model is not justified.

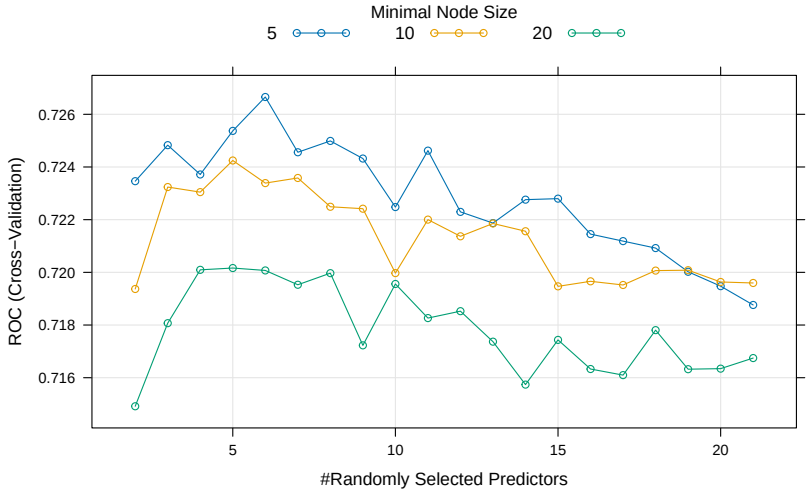


Figure 3: Cross-validation plot for random forest. The hyperparameters are minimum number of nodes (s) and number of features randomly selected (d). The best performing model has s = 5 and d = 6. However, it's predictive performance does not justify complexity so a model with s = 20 and d = 4 is selected.

Gradient Boosting Machine

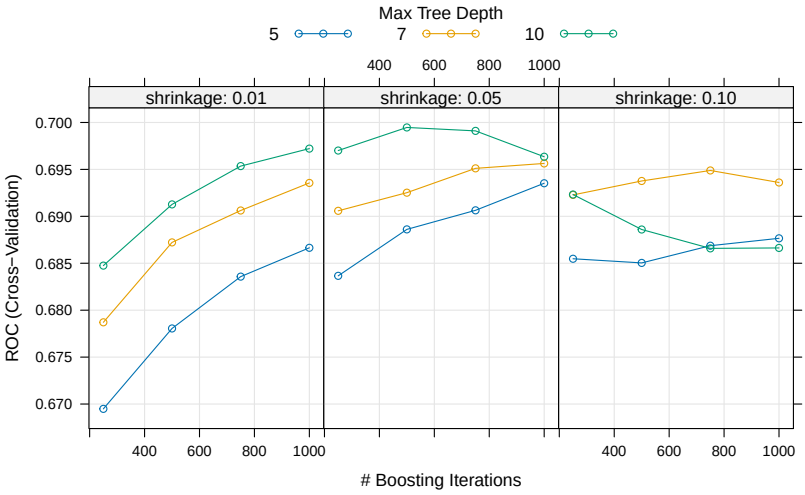


Figure 4: Hyperparameter tuning profile for Gradient Boosting Machine

Interpretation:

Shrinkage rates = 0.05:

- At low shrinkage rate `0.01`, the ROC curves show a consistent, steady upward trajectory. This indicates that the model is learning slowly and has not yet plateaued. While stable, it requires significantly more iterations to reach peak performance. In contrast, at high shrinkage rate `0.1`, the performance is significantly worse and highly unstable (notice the crossing lines in the third panel). This suggest that a learning rate of 0.10 is too aggressive for this data set, causing the model to overfit the training folds too quickly and fail to generalize during cross-validation.
- The optimal zone is observed at moderate shrinkage rate at `0.05`. The curve reach high ROC values (peaking near 0.7) and begin to level off, suggesting the model has captured most of the signal wihout being erratic.

Max Tree Depths = 10 (Interaction Effects):

- The depth at 10 (green) outperforms depths 5 and 7 across 0.05 shrinkage panel. This indicates that the High Availability of property listings is driven by complex interations rather than simple, additive features. The depth at 5 showest the lowest performance. This suggest likely underfitting the data, failing ot capture the nuances of the property market.

Boosting Iterations = 500:

- Given the select optimal combination (`shrinkage=0.05`, `depth=10`), the peak performance is at 500 iterations. After 500 iterations, the ROC for depth 10 begins to decline. This suggests overfitting, beyond 500 trres, the model starts memorizing noise in the training data rather than learning general patterns.

K-Nearest Neighbors

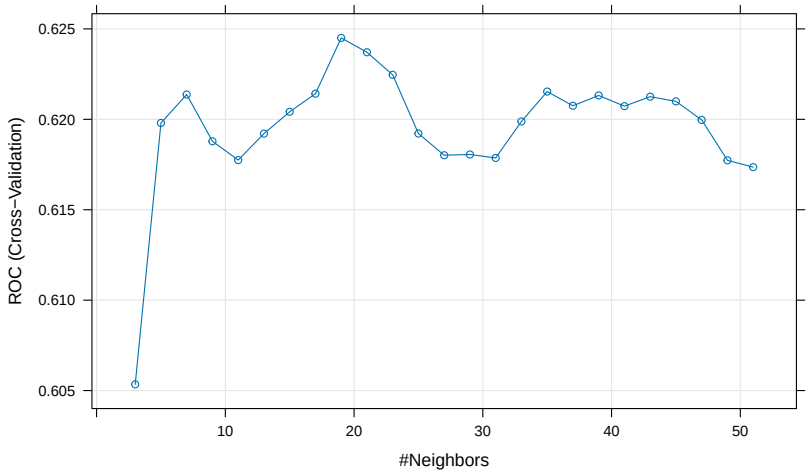


Figure 5: Hyperparamter tuning profile for K-Nearest Neigbors (KNN)

Interpretation:

K = 19:

- At very low values of k (e.g. $k < 5$), the ROC is at its lowest point (~0.066). In this region, the mdel is overly sensitive to local noise and individual outliers in the Cape Town propery dataset, resulting in a high-variance classifier that fails to generalize to the validation folds.
- At the global optimum ($k = 19$), the Area Under the ROC Curve (AUROC) reaches its maximum peak of approximately 0.625. This is where the neighborhood is large enough to smooth out random fluctuations in listing availability but small enough to maintain the local geographic and host-specific signals.
- Beyond $k = 19$, the performance remains relatively stable, oscillating between 0.618 and 0.622. The minor peaks at $k = 35$ and $k = 45$ suggest that the underlying decision boundary is somewhat robust, but the slight downward trend toward $k = 50$ indicates the onset of oversmoothing i.e. high bias.

Model Comparison

Table 1 compares the cross-validated performance of the four models using accuracy, F1 score, recall, specificity and AUROC. The random forest has the best performance, with the largest AUROC (0.720) and the highest F1 score (0.817), showing the best balance between precision and recall. XGBoost performs similarly, but with slightly lower AUROC (0.699), while elastic-net logistic regression and k-nearest neighbours show weaker classification ability, with AUROC values close to 0.60–0.63. All models show very high recall but relatively low specificity, which indicates that the classifiers tend to predict the "High" availability class more often than the "Low" class. This behaviour is consistent with the class distribution in the data, where a larger proportion of listings belong to the high-availability category The random forest provides the best trade-off between predictive performance and stability, and is therefore selected for further analysis.

Table 1: Comparative Performance Metrics across Logistic (Elastic-Net), Random Forest, XGBoost, and KNN models. Values represent cross-validated averages from the 10-fold tuning process.

Model	Accuracy	F1	Recall	Specificity	AUROC
Elastic-Net	0.67400	0.80400	0.9950	0.01100	0.602
RandomForest	0.71412	0.81714	0.9477	0.23121	0.720
XGBoost	0.70500	0.80600	0.9070	0.28800	0.699
KNN	0.67400	0.79000	0.9100	0.18600	0.625

3 Threshold Optimisation

The F1 score is a measure of how well a model balances precision and recall. It provides a better assessment of predictive ability than accuracy, which does not account for class imbalance, as is the case with the data under study. When cross-validation was performed previously, τ (the threshold) was 0.5. Figure 6 shows cross-validation results for the random forest with $s = 20$ and $d = 4$, as hyperparameter τ varies with metric F1 score. The highest F1 score is 0.819 which occurs at $\tau = 0.467$. The plot shows that the F1 score for values of τ below 0.467 are close to the maximum with negligible differences. This suggests that many observations have moderately high predicted probabilities, around 0.5 to 0.6. This is consistent with observations made on the data before modelling, as it was observed that many listings display characteristics associated with professional hosting. Thus, even misclassifying some listings as "High" at lower thresholds has almost no effect on predictive performance.

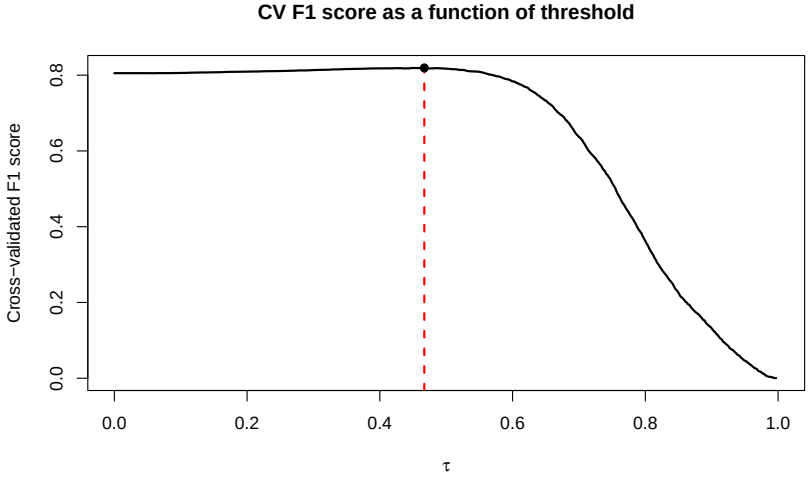


Figure 6: Cross-validated F1 score as a function of the classification threshold. The threshold = 0.47 corresponds to the highest F1 score 0.819.

The ROC curve for the selected random forest model is shown in Figure 7. The curve is above the diagonal showing that the model has some ability to distinguish between high and low availability, so it is better than random classification. This curve is associated with an AUC of 0.72 showing moderate predictive ability. The point associated with threshold that results in the highest F1 score is shown as a red point on the plot. At this threshold, the model places high importance on classifying true positives over wrongfully classifying observations as positive. This is illustrated by a high true positive and false positive rate at this threshold (0.47). Models that are good classifiers will be closer to the top left corner of this plot, whereas the best performing model based on F1 score is very distant from this area. Moving to another threshold value would be a trade-off between true positive and false positive rate. This is consistent with results seen in Table (table) where the model showed low specificity.

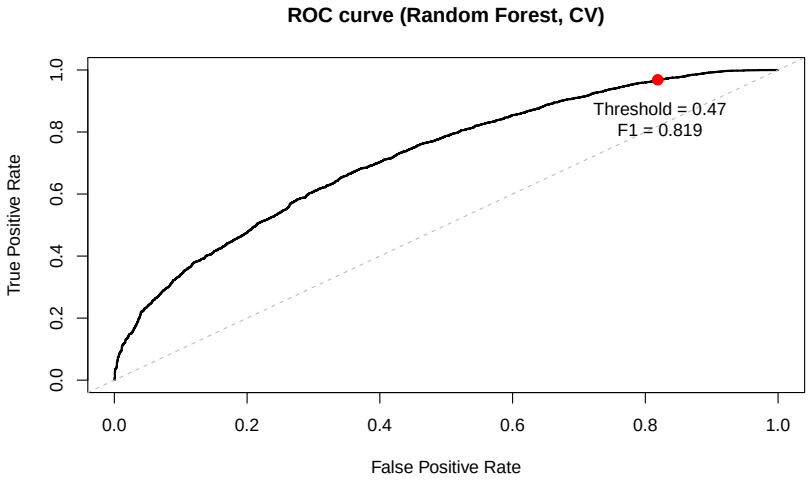


Figure 7: ROC curve for the selected random forest model based on cross-validated predictions. The red point indicates the threshold (0.47) that maximises the F1 score (0.819).

Table 2 compares the performance of the selected random forest model at the optimal threshold for both the cross-validation and test data. The results are very similar, with slightly higher accuracy, F1 score and AUROC on the test set, which suggests that the model performs consistently and does not appear to overfit the training data. The recall values are high in both cases, showing that the model is able to correctly identify most listings with high availability. However, the specificity is relatively low, indicating that the model often classifies low-availability listings as high availability. This is likely due to the fact that most observations in the data belong to the high-availability class, making it harder for the model to correctly identify the smaller class. The AUROC values around 0.72–0.75 indicate moderate predictive ability, meaning that the model can distinguish between the two classes reasonably well but not perfectly. Overall, the random forest shows stable and reliable performance on both datasets.

Table 2: Random forest performance at the optimal threshold

Data	Accuracy	F1	Precision	Recall	Specificity	AUROC
Cross-validation	0.714	0.817	0.718	0.948	0.231	0.720
Test	0.721	0.825	0.720	0.967	0.190	0.745

4 Interpretation and Error Analysis

Variable Importance Plots

To better understand the key drivers of listing availability, variable importance measures were computed using the fitted Random Forest model.

Two complementary approaches were considered:

1. Impurity-based importance: reflects how frequently and effectively variables are used to split the data.
2. Permutation importance: evaluate the impact of each variable on predictive.

While impurity importance provides insight into the internal structure of the model, permutation importance offers a more reliable assessment of true predictive contribution. Therefore, will be prioritized in the analysis.

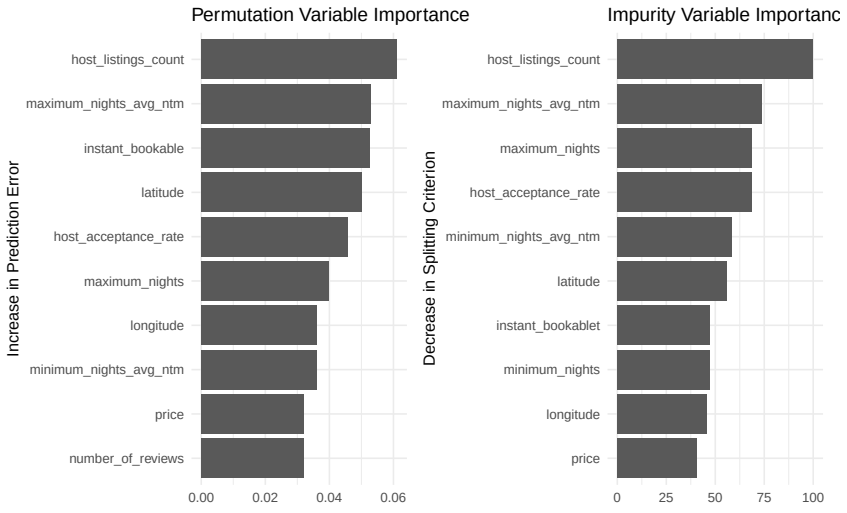


Figure 8: Comparative Variable Importance for the Random Forest Model. The left panel shows Permutation Importance (impact on accuracy), while the right panel shows Impurity Importance (Gini gain)

Interpretation:

- Based on permutation importance, `host_listing_count` emerges as the dominant variable. This indicates that the scale of host activity play a central role, with larger, more commercial hosts exhibiting systematically different availability patterns compared to smaller hosts.
- The next most influential predictors include `maximum_nights_avg_ntm` and `instant_bookable`, both of which capture aspects of booking flexibility and convenience. This suggests that availability is strongly shaped by how easily and flexibly a listing can be booked. Listings with more permissive booking policies and reduced friction tend to display higher availability, highlighting of host strategy in determining outcomes.
- Spatial variables, name `latitude` and `longitude`, also contribute meaningfully to the model. Their presence among the top predictor indicates that availability is not uniform across regions, but instead reflects underlying geographic variation in demand. This aligns with expectations in short-term rental markets, where location remain a key determinant of performance.
- In contrast, variables traditionally associated with demand, such as `price` and `number_of_reviews`, exhibit comparatively lower importance. This suggest that, withing this dataset, structural and host-related factors outweigh pure demand signals in explaining availability.

Overall, the consistency between permutation and impurity-based importance for top variables strengthens confidence in these findings. Although, some divergence in lower-ranked predictors highlights the known bias of impurity measures.

Partial Dependence Plots

Building on the variable importance results, partial dependence plots were used to further investigate how the most influential predictors affect the probability of high availability.

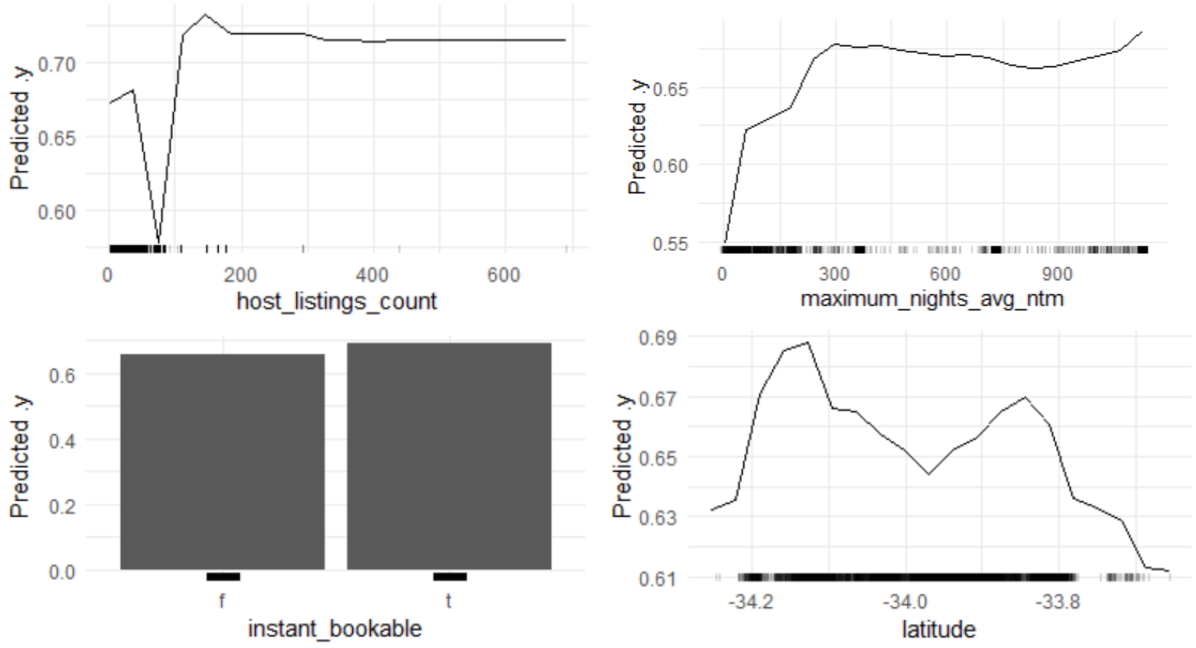


Figure 9: Partial Dependence Plots (PDP) for the four most influential predictors of High Availability

Interpretation:

- The partial dependence plot for `host_listing_count` exhibits a sharp decline in high availability for host with roughly 50-100 listings, followed by a dramatic surge. This suggests that mid-tier managers may have higher occupancy i.e. lower availability. However, once a host crosses the professional threshold (approximately 150+ listings), availability stabilizes at high level (70%+). This suggests that large-scale commercial operators maintain high "buffer" inventory or have different listing strategies than mid-sized ones.
- The partial dependence plot for `maximum_night_avg_ntm` illustrates probability of high availability climbs steeply as the maximum stay limit increases from 0 to 300 nights, after which it plateaus. This suggest that listings that allow long-term stays (300+ nights) are significantly more likely to be classified as high availability. This implies these listings are likely targeting the medium-to-long-term rental market rather than high-turnover short-stay tourists.
- For `latitude`, the plot shows a distinct bi-modal distribution. Availability peaks at appropriately -35.15 and -33.85, with a significant trough (lower availability/higher demand) around -34.0. In the context of Cape Town, the trough around -34.0 likely correspond to prime high-demand areas (e.g. City Bowl or Atlantic Seaboard) where occupancy is highest, while peaks represent suburban or outlying areas where listings sit vacant more often.
- The Partial Dependence Plot for `instant_bookable` reveals that while the variable is a significant predictor in the Random Forest model, its marginal impact on the probability of high availability is negligible. In the Cape Town context, this indicates that highly available listings are distributed almost equally across both booking styles, suggesting that "power hosts" and casual hosts alike maintain high availability regardless of their preference for manual guest screening.

Missclassifcations

Across the different false positives and false negatives, a consistent pattern can be observed. Listings that are incorrectly classified as high availability often display characteristics associated with professional hosting, such as higher maximum night limits, larger host listing counts, higher acceptance rates, or the use of instant booking. These features were identified earlier as important predictors in the random forest model, which explains why the model tends to classify such listings as high availability even

when they are not. In contrast, false negatives tend to correspond to listings with fewer host listings, lower maximum night limits, and less flexible booking settings. Although these listings are truly high availability, their characteristics resemble those of low-availability properties, making them difficult for the model to distinguish correctly.

Another noticeable pattern is that many misclassified observations have moderate values for the most important variables rather than extreme values, suggesting that they lie close to the decision boundary of the classifier. This is consistent with the moderate AUROC obtained earlier, which indicates that the predictors contain useful but not perfectly separating information. Overall, the misclassifications suggest that the model relies strongly on indicators of professional hosting and booking flexibility when predicting availability, and errors mainly occur for listings that do not clearly fit into either the high- or low-availability profile.

Table 3: Five false positives from the test set

	Max nights	Host count	Instant	Accept_rate	Lat	True	Pred
8199	1125	1	f	100	-33.88210	Low	High
9241	1125	17	f	84	-33.93463	Low	High
3469	1125	1	f	95	-33.91712	Low	High
226	1125	5	t	99	-33.89102	Low	High
3874	365	3	t	77	-33.89264	Low	High

Table 4: Five false negatives from the test set

	Max nights	Host count	Instant	Accept_rate	Lat	True	Pred
8913	20	1	f	86	-33.98033	High	Low
9599	25	1	f	82	-34.04358	High	Low
7200	93	3	f	54	-33.91767	High	Low
3601	180	5	f	55	-33.92568	High	Low
5853	21	1	f	100	-34.00788	High	Low